

Chat via UDP: Confirmação de Entrega e Reenvio Manual em Nível de Aplicação

Relatório da Atividade Prática

Disciplina: Redes de Computadores

1 Estratégia Adotada

A aplicação foi dividida em dois papéis complementares, ambos operando exclusivamente sobre sockets UDP (`SOCK_DGRAM`), sem qualquer garantia de entrega nativa do transporte.

No lado do **remetente** (`chat_sender.py`), cada mensagem digitada pelo usuário recebe um identificador sequencial único (`msg_counter`) e é armazenada em um dicionário de controle, `pending_messages`, no formato `{id: texto}`, antes mesmo de ser enviada ao destinatário. Essa estrutura funciona como uma "lista de pendências": enquanto o ID permanecer nela, a mensagem é considerada *não confirmada*. O envio é feito no formato `MSG|<ID>|<CONTEUDO>`, conforme especificado no protocolo de aplicação.

Uma *thread* separada (`listen_receipts`) fica dedicada exclusivamente a escutar o socket em busca de recibos vindos do servidor, no formato `DELIVERED|<ID>`. Ao receber um recibo válido, o ID correspondente é removido de `pending_messages`, e o status da mensagem passa de "Pendente" para "Entregue". O uso de uma thread separada permite que o recebimento dos recibos ocorra de forma assíncrona, sem bloquear a leitura de novas mensagens digitadas pelo usuário no terminal. Um `Lock` (`threading.Lock()`) é utilizado para evitar condições de corrida no acesso concorrente ao dicionário compartilhado entre a thread principal e a thread de escuta.

Dois comandos especiais foram implementados para dar visibilidade e controle ao usuário:

- `/status`: percorre `pending_messages` e exibe quantas mensagens ainda aguardam confirmação, junto com seus respectivos IDs e conteúdos;
- `/reenviar`: itera sobre todas as mensagens ainda pendentes e as reenvia integralmente ao servidor, simulando uma retransmissão manual em nível de aplicação – já que o UDP, por si só, não reenviaria nada automaticamente em caso de perda.

No lado do **destinatário** (`chat_receiver.py`), cada datagrama recebido é submetido a um descarte artificial com probabilidade de 40% (`DROP_RATE = 0.4`), simulando a perda de pacotes em um canal não confiável. Quando o pacote não é descartado, a mensagem é decodificada, o separador `|` é utilizado para extrair o tipo, o ID e o conteúdo, a mensagem é exibida no terminal e um recibo `DELIVERED|<ID>` é enviado de volta ao endereço de origem, confirmando o recebimento na camada de aplicação.

Dessa forma, todo o controle de confiabilidade – identificação de mensagens, confirmação de entrega e retransmissão – é implementado manualmente pela aplicação, e não delegado ao protocolo de transporte.

2 Captura de Tela 1 – Envio de Mensagens com Descarte Proposital

A Figura 1 mostra o cliente enviando mensagens ao servidor. É possível observar no terminal do servidor (esquerda) a mensagem [CANAL] Pacote descartado artificialmente!, indicando que o canal simulado descartou um pacote antes que ele fosse processado. No terminal do cliente (direita), o comando `/status` é utilizado para exibir as mensagens que ainda aguardam confirmação de entrega ([PENDENTE]), evidenciando que o mecanismo de controle de pendências está funcionando corretamente mesmo diante da perda simulada de pacotes.

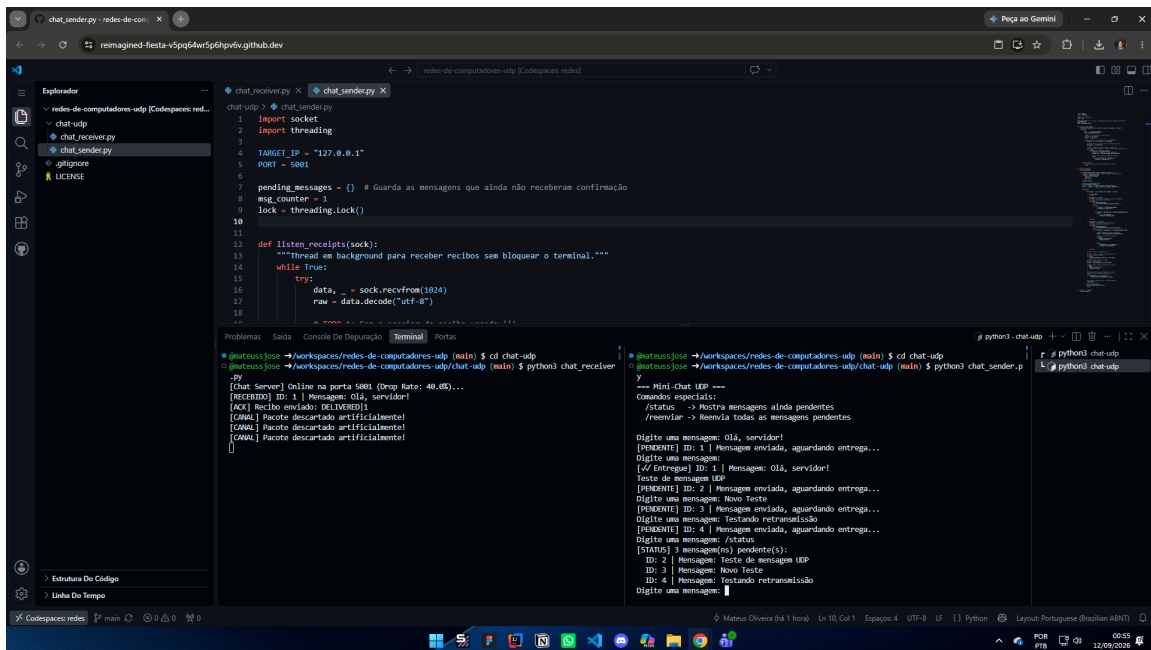


Figura 1: Cliente enviando mensagens com descarte proposital pelo canal e status pendente visível via `/status`.

3 Captura de Tela 2 – Execução do Comando `/reenviar`

A Figura 2 apresenta a execução do comando `/reenviar`. Nesse momento, todas as mensagens que ainda constavam em `pending_messages` são retransmitidas ao servidor. É possível observar as mensagens [REENVIADO] no terminal do cliente, seguidas pela chegada dos recibos correspondentes, identificados por [Entregue]. Ao final, o comando `/status` confirma que não há mais mensagens pendentes ([STATUS] Nenhuma mensagem pendente.), demonstrando que o mecanismo de reenvio manual foi capaz de destravar e confirmar a entrega de todas as mensagens que haviam sido perdidas anteriormente.

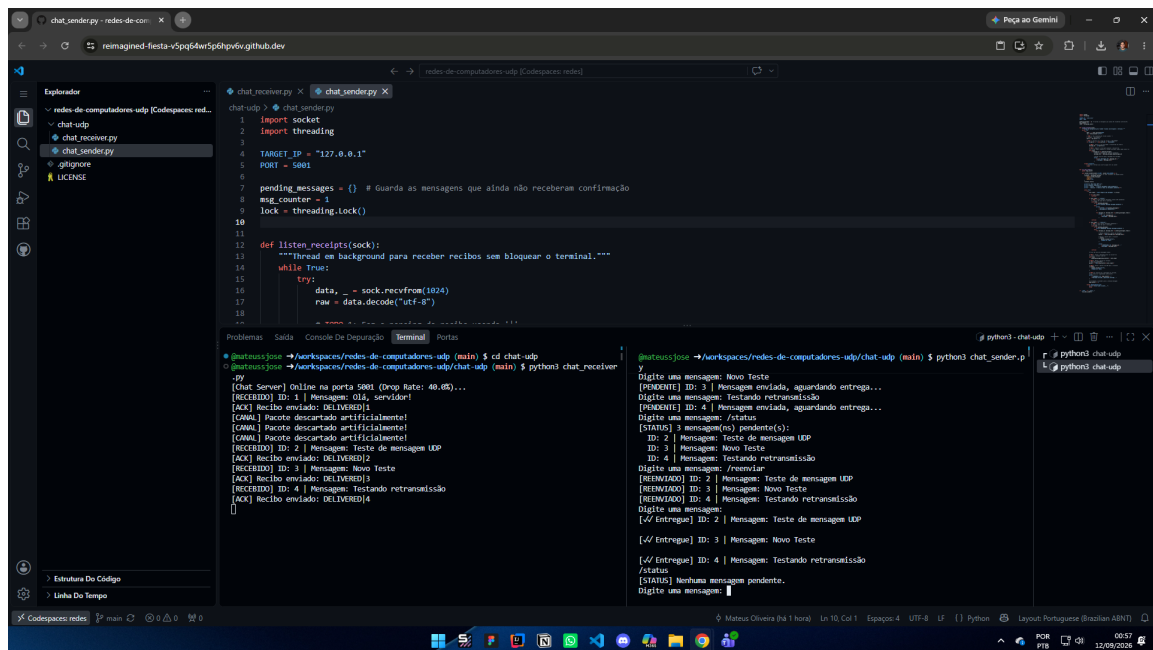


Figura 2: Execução do comando `/reenviar`, destravando e confirmando a entrega de todas as mensagens pendentes.

4 Discussão: Controle na Camada de Aplicação versus Confiabilidade Nativa do TCP

A implementação realizada nesta atividade evidencia, na prática, o esforço que normalmente fica oculto quando se utiliza o TCP. No TCP, a confiabilidade é uma responsabilidade da própria camada de transporte: o protocolo estabelece conexão via *three-way handshake*, numera segmentos, confirma o recebimento por meio de ACKs, retransmite automaticamente segmentos perdidos após *timeout* ou ACKs duplicados, reordena segmentos que chegam fora de ordem e ainda realiza controle de fluxo e de congestionamento. Para a aplicação, tudo isso é transparente: basta escrever e ler em um fluxo contínuo de bytes, sem se preocupar com perdas ou duplicações.

No UDP, nenhuma dessas garantias existe. O protocolo opera sob o paradigma de melhor esforço, entregando datagramas de forma independente e sem qualquer confirmação. Isso significa que toda a lógica de confiabilidade – que no TCP é resolvida uma única vez, de forma genérica, dentro do sistema operacional – precisa ser reimplementada manualmente em cada aplicação que dela necessite. Neste projeto, isso se traduziu na necessidade de: (i) definir um formato próprio de mensagens com identificadores únicos; (ii) manter uma estrutura de controle de pendências; (iii) implementar uma thread dedicada para escutar recibos de forma assíncrona; e (iv) criar um mecanismo manual de retransmissão, acionado explicitamente pelo usuário via `/reenviar`, em vez de ocorrer automaticamente por *timeout* como acontece no TCP.

Essa abordagem tem vantagens e desvantagens. Por um lado, o UDP oferece menor overhead e maior controle sobre o comportamento da aplicação, permitindo, por exemplo, decidir que uma mensagem antiga não vale mais a pena ser retransmitida – algo que seria mais difícil de customizar no TCP. Por outro lado, o custo de desenvolvimento e manutenção aumenta consideravelmente, pois a aplicação passa a ser responsável por

tratar cenários de perda, duplicação e reordenação que o TCP resolveria de forma nativa e transparente. Essa é, inclusive, a razão pela qual diversos protocolos modernos (como o QUIC, usado no HTTP/3) optam por implementar sua própria camada de confiabilidade sobre o UDP: buscam o baixo overhead do UDP aliado a mecanismos de confiabilidade customizados, exatamente como foi explorado nesta atividade em pequena escala.